

Module 1 Activity 1: Python Basics

Part 1 | Variables

A variable in Python is a name that is a reference or pointer to an object. Once an object is assigned to a variable, you can refer to the object by that name. Variables are essential in programming because they allow you to store, modify, and use data dynamically.

Rules for Naming Variables

1. Variable names must start with a letter (a-z, A-Z) or an underscore (_).
2. The rest of the name can include letters, numbers, and underscores.
3. Variable names are case-sensitive (**age**, **Age** and **AGE** are different variables).
4. Avoid using reserved words or built-in function names as variable names (like **print**, **if**, **else**, etc.).

Examples of Variables in Python

1. Assigning a Value to a Variable

```
# Assigning integer value to a variable
age = 25
# Assigning string value to a variable
name = "Alice"
# Assigning float value to a variable
height = 5.7
# Assigning boolean value to a variable
is_student = True

print(age)          # Output: 25
print(name)         # Output: Alice
print(height)       # Output: 5.7
print(is_student)   # Output: True
```

2. Using Variables in Expressions

```
# Using variables in arithmetic operations
x = 10
y = 5
sum = x + y           # sum is now 15
difference = x - y    # difference is now 5
product = x * y       # product is now 50
quotient = x / y      # quotient is now 2.0

print(sum)            # Output: 15
print(difference)     # Output: 5
print(product)        # Output: 50
print(quotient)       # Output: 2.0
```

3. Variables and Strings

```
# Concatenating strings using variables
first_name = "John"
last_name = "Doe"
full_name = first_name + " " + last_name

print(full_name)      # Output: John Doe

# Using f-strings for formatting
age = 30
greeting = f"Hello, my name is {full_name} and I am {age} years old."

print(greeting)       # Output: Hello, my name is John Doe and I am 30 years old.
```

Next Steps

Explore variables in more detail and store different values in the variables. Variables are fundamental to programming in Python and will allow you to write flexible and dynamic code by storing and manipulating data. We will be using variable extensively throughout the course.

Part 2 | Quotes

In Python, quotes are used to define strings. There are three types of quotes you can use: single quotes ('), double quotes ("), and triple quotes (''' or """). Each type of quotes has its own use cases and advantages.

1. Single Quotes (')

Single quotes are used to create string literals. They are typically used for short strings and when the string itself does not contain single quotes.

```
# Single quotes
message = 'Hello, World!'
print(message) # Output: Hello, World!
```

2. Double Quotes (")

Double quotes are also used to create string literals. They are particularly useful when the string contains single quotes, such as in contractions or possessives.

```
# Double quotes
message = "It's a beautiful day!"
print(message) # Output: It's a beautiful day!
```

3. Triple Quotes (''' or """)

Triple quotes are used for multi-line strings or docstrings (comments that describe functions, classes, or modules). They allow you to include both single and double quotes within the string without escaping them.

```
# Triple single quotes for multi-line string
multi_line_message = '''This is a string
that spans multiple
lines.'''
print(multi_line_message)
# Output:
# This is a string
# that spans multiple
# lines.
```

```
# Triple double quotes for multi-line string
another_message = """You can also use
triple double quotes
for multi-line strings."""
print(another_message)
# Output:
# You can also use
# triple double quotes
# for multi-line strings.
```

Escape Characters

When you need to include special characters (like quotes) inside a string defined by the same type of quotes, you can use escape characters (preceded by a backslash \).

```
# Using escape characters
message_with_quotes = 'She said, "Hello!'"
print(message_with_quotes) # Output: She said, "Hello!"

# Including single quotes in a single-quoted string
another_message_with_quotes = 'It\'s a beautiful day!'
print(another_message_with_quotes) # Output: It's a beautiful day!

# Including double quotes in a double-quoted string
yet_another_message_with_quotes = "She said, \"Hello!\""
print(yet_another_message_with_quotes) # Output: She said, "Hello!"
```

Raw Strings

Raw strings treat backslashes (\) as literal characters, which is useful for regular expressions, file paths, and when you want to avoid escaping backslashes.

```
# Raw string
raw_string = r"C:\Users\Name\Documents"
print(raw_string) # Output: C:\Users\Name\Documents
```

String Concatenation

You can concatenate strings using the + operator.

```
# String concatenation
first_part = 'Hello'
second_part = 'World'
full_message = first_part + ', ' + second_part + '!'
print(full_message) # Output: Hello, World!
```

String Formatting

There are several ways to format strings, the f-strings (formatted string literals) is used with Python 3.6+ which will be used in this course.

```
name = "Bob"
age = 25
formatted_message = f"My name is {name} and I am {age} years old."
print(formatted_message) # Output: My name is Bob and I am 25 years old.
```

Understanding how to use quotes effectively allows you to handle string data more flexibly and intuitively in Python.

Part 3 | Indexing

Indexing in Python is a way to access individual elements of a sequence-like data structure, such as strings, lists, or tuples. Each element in a sequence is assigned a unique index starting from **0**. You can use these indices to retrieve, modify, or manipulate the elements within the sequence.

1. Indexing in Strings

Strings in Python are sequences of characters, and you can access individual characters using their index.

```
# Example string
text = 'Hello, World!'

# Accessing characters using positive indices
print(text[0])    # Output: H
print(text[7])    # Output: W
print(text[12])   # Output: !

# Accessing characters using negative indices
print(text[-1])   # Output: !
print(text[-7])   # Output: W
print(text[-13])  # Output: H
```

2. Indexing in Lists

Lists are ordered collections of items, and you can access any item using its index.

```
# Example List
numbers = [10, 20, 30, 40, 50]

# Accessing elements using positive indices
print(numbers[0]) # Output: 10
print(numbers[2]) # Output: 30
print(numbers[4]) # Output: 50

# Accessing elements using negative indices
print(numbers[-1]) # Output: 50
print(numbers[-3]) # Output: 30
print(numbers[-5]) # Output: 10
```

4. Slicing

Slicing allows you to access a range of elements in a sequence.

```
# Example string
text = 'Hello, World!'

# Slicing a string
print(text[0:5])    # Output: Hello (from index 0 to 4)
print(text[7:12])   # Output: World (from index 7 to 11)
print(text[:5])     # Output: Hello (from start to index 4)
print(text[7:])     # Output: World! (from index 7 to end)
print(text[::-2])   # Output: Hlo ol! (every second character)

# Example list
numbers = [10, 20, 30, 40, 50]

# Slicing a list
print(numbers[1:4]) # Output: [20, 30, 40] (from index 1 to 3)
print(numbers[:3])  # Output: [10, 20, 30] (from start to index 2)
print(numbers[2:])  # Output: [30, 40, 50] (from index 2 to end)
print(numbers[::-2]) # Output: [10, 30, 50] (every second element)
```

5. Modifying Lists Using Indexing

Unlike strings and tuples, lists are mutable, meaning you can change their elements.

```
# Example List
numbers = [10, 20, 30, 40, 50]

# Modifying elements using indices
numbers[0] = 15
print(numbers) # Output: [15, 20, 30, 40, 50]

numbers[-1] = 55
print(numbers) # Output: [15, 20, 30, 40, 55]

# Modifying elements using slicing
numbers[1:3] = [25, 35]
print(numbers) # Output: [15, 25, 35, 40, 55]
```

Part 4 | Importing Modules

Import statements are used to include the definitions and functionalities of other modules into your current script. This allows you to leverage existing libraries, organize your code better, and avoid redundancy.

1. Basic Import

The simplest form of import statement is used to import an entire module.

```
import math

# Using functions from the math module
result = math.sqrt(16)
print(result) # Output: 4.0
```

2. Importing a Module with an Alias

Using aliases when importing modules can make your code more readable and concise. The alias can be defined by the user.

If you are working with map documents and layers, you can import the mapping module with an alias.

```
import arcpy.mp as MAP

# Reference a map project
aprx = MAP.ArcGISProject(r"C:\path\to\your\map.aprx")
```

4. Importing All Definitions

You can import all definitions from a module using the * operator.

```
import arcpy
from arcpy.sa import *

# Check out the Spatial Analyst extension
arcpy.CheckOutExtension("Spatial")

# Set the workspace environment
arcpy.env.workspace = r"C:\path\to\your\workspace"

# Perform a slope calculation
in_raster = "elevation.tif"
out_raster = "slope.tif"
slope_raster = Slope(in_raster)
slope_raster.save(out_raster)

# Check in the Spatial Analyst extension
arcpy.CheckInExtension("Spatial")
```

The examples provided in this activity will be used frequently throughout the course. Be familiar with each of these and comfortable with their functionalities.